NATIONAL UNIVERSITY

NATIONAL UNIVERSITY
COLLEGE OF COMPUTING AND
INFORMATION TECHNOLOGIES

Laboratory Activity 5
Custom Middleware and API Security

Name	Marzan, Joshua M.
Section	INF 233
Date	20 October 2026

Objectives

- Explain the role of middleware in an Express.js application.
- Develop custom middleware for processing and validating API requests.
- Implement request validation for Bulldogs Exchange API resources.
- Implement centralized error handling for API requests.
- Configure Helmet, CORS, and rate limiting to improve API security.
- Test the secured Bulldogs Exchange APIs using Postman.

Materials Needed:

- Visual Studio Code
- Node.js (LTS Version)
- MongoDB Community Server or MongoDB Atlas
- Postman/Insomnia REST

Prerequisite(s):

- Completed Week 5 – Authentication and Authorization
- Working Bulldogs Exchange REST API
- JWT authentication and RBAC already implemented
- Product, Category, and Supplier APIs functioning correctly

Submission Requirements:

- Filename: Surname_Activity4_CTADWEBL
- Save as PDF

Activity Instructions

Part 1: Running the Server
Open the Bulldogs Exchange backend project completed during Week 4. **Verify that (***Provide the Screenshot of your Backend Working***):** • The server starts successfully. • MongoDB connects successfully. • Existing RESTful APIs are functioning.

The Bulldogs Exchange backend (**marzan-server**) was started with `npm run dev`. The console confirms the MongoDB Atlas connection and that every Mongoose schema, now including the new `logModel`, loaded without errors, and that the RESTful APIs and RBAC from Activities 3–4 still respond correctly underneath the new security middleware added in this activity.
`> npm run dev`
`MongoDB connected`
`Models loaded: Log, Product, Category, Supplier, User, Cart, Order, Review`
`Server running on http://localhost:5000`
• **The server starts successfully:** `GET http://localhost:5000/` returns `{ "success": true, "message": "BulldogEx Shop API is running", ... }`.
• **MongoDB connects successfully:** the console prints `MongoDB connected` (database `bulldogex-shop` on MongoDB Atlas).
• **Existing RESTful APIs are functioning:** `GET /api/v1/products` (public, with pagination/filter/sort from Activity 3) and `POST /api/v1/sessions` as Admin (JWT + RBAC from Activity 4) both still return the expected results with Helmet, CORS, the audit logger, and the rate limiter now sitting in front of them.
Screenshot below: VS Code with the terminal output of `npm run dev`.

Part 2: Creating Custom Middleware

Create the following files inside the middleware folder (*Provide Screenshots of the Directory***):**

- **auditLoggerMiddleware.js**
- **validationMiddleware.js**
- **rateLimiterMiddleware.js**

Three files were created inside `components/middleware/`, exactly as named in the activity:

- **auditLoggerMiddleware.js** — logs every request (Part 3–4).
- **validationMiddleware.js** — validates request bodies before they reach a controller (Part 5–6).
- **rateLimiterMiddleware.js** — rate limits and the login lockout (Part 7–9).

Two small supporting pieces were also added, following the project's existing pattern of giving every Mongoose model its own controller and route (Activity 2 Part 9): `components/models/logModel.js` (what the audit logger writes to) and `components/controllers/logController.js` + `components/routes/logRoutes.js` (an Admin-only `GET /api/v1/logs`, so the audit log can be checked straight from Postman instead of switching to MongoDB Atlas for every test).

Screenshot below: VS Code Explorer showing components/middleware (and components/models/logModel.js).

Part 3: Creating an Audit Logger Middleware

Do the following (*Provide Screenshot of your Source Cod***):**

- Before starting up test how your server typically returns its response.
- Identify what should your audit log includes.
- After identifying it create your own audit logger
- You can refer below for the sample source code

Example Code:

```
const Log = require("../model/logModel.js");

const logger = {
  info: async (message, metadata = {}) => {
```

```
    try {
      await Log.create({ level: "info", message, metadata });
    } catch (error) {
      console.error("Failed to write info log:", error);
    }
  },

  warn: async (message, metadata = {}) => {
    try {
      await Log.create({ level: "warn", message, metadata });
    } catch (error) {
      console.error("Failed to write warn log:", error);
    }
  },

  error: async (message, metadata = {}) => {
    try {
      await Log.create({ level: "error", message, metadata });
    } catch (error) {
      console.error("Failed to write error log:", error);
    }
  }
};

// Middleware to log all requests
const auditLoggerMiddleware = async (request, response, next) => {
  const startTime = Date.now();

  // Intercept response to log status code
  const originalSend = response.send;
  response.send = function (data) {
    response.send = originalSend;

    const duration = Date.now() - startTime;

    // Log the request (after authentication has run)
    const logData = {
      method: request.method,
      path: request.path,
      ip: request.ip || request.connection.remoteAddress,
      userId: request.user?.userID || null,
      username: request.user?.username || null,
      statusCode: response.statusCode,
      metadata: { duration: `${duration}ms` }
    };

    // Log based on status code
```

```
  const message = `${logData.method} ${logData.path} - ${logData.statusCode}`;
  if (response.statusCode >= 500) {
    logger.error(message, logData);
  } else if (response.statusCode >= 400) {
    logger.warn(message, logData);
  } else {
    logger.info(message, logData);
  }

  return response.send(data);
 };

 next();
};

module.exports = { logger, auditLoggerMiddleware };
```

Before building the logger, the server's typical response was tested (Activities 2–4): every response already follows one shape, `{ success, message, data }` (or `{ success, message }` on error), built by `res.json()` through the `sendSuccess`/`sendError` helpers in `utils/apiResponse.js`. That matters because the logger has to intercept however the response is actually sent, not assume `res.send()` is called directly.

What the audit log includes

- **method** and **path** — which endpoint was called.
- **ip** — where the request came from.
- **userId, email, role** — who made it (`null` for anonymous requests, read from `req.user`, set by `authentication.js` on protected routes).
- **statusCode** — how the request ended (also decides the log level: info < 400, warn 400–499, error 500+).
- **duration** — how long the server took to respond, useful for spotting slow endpoints.

components/models/logModel.js

```
const logSchema = new mongoose.Schema({
  level: { type: String, enum: ["info", "warn", "error"], required: true },
  message: { type: String, required: true },
  metadata: { type: mongoose.Schema.Types.Mixed, default: {} },
}, { timestamps: true });
logSchema.index({ createdAt: -1 });
```

components/middleware/auditLoggerMiddleware.js

Follows the reference code's design (a `logger.info/warn/error` helper writing to the `Log` model, and middleware that wraps `response.send` to capture the final status code and duration) with two deliberate changes, found while testing this against the project's own nested routers:

- **`request.originalUrl` instead of `request.path`.** Express temporarily strips the mount-point prefix from `req.path`/`req.url` while a request is inside a nested router (e.g. `categoryRoutes`), and only restores it after the handler calls `next()` — but a terminal handler (ours) never calls `next()`, so by the time `response.send()` runs, `req.path` was still showing `/:id` instead of

`/api/v1/categories/:id`. `originalUrl` is never rewritten, so it was used instead.

- **The log write is `await`ed before the response is sent**, instead of fire-and-forget like the example. This guarantees the log already exists by the time the client (or a Postman test right after it) sees the response, at the cost of a few extra milliseconds per request — worth it so Part 4's "verify each request produces a log" is actually reliable to test.

```
const originalSend = response.send;
response.send = async function (data) {
  response.send = originalSend;
  const logData = {
    method: request.method,
    path: request.originalUrl, // not request.path - see note above
    ip: request.ip,
    userId: request.user?.id || null,
    email: request.user?.email || null,
    role: request.user?.role || null,
    statusCode: response.statusCode,
    metadata: { duration: `${Date.now() - startTime}ms` },
  };
  const message = `${logData.method} ${logData.path} - ${logData.statusCode}`;
  if (response.statusCode >= 500) await logger.error(message, logData);
  else if (response.statusCode >= 400) await logger.warn(message, logData);
  else await logger.info(message, logData);
  return response.send(data);
};
```

Screenshot below: components/middleware/auditLoggerMiddleware.js.

Part 4: Implementing your Audit Logger

Do the following (*Provide Screenshot of the Source code and Output, Add more pages if necessary***):**

- Add your Audit Logger to your server.js file
- Verify that each request produces an auditLog
- Perform several functions (CRUD) using Postman/Insomnia REST

- Refer to the Example Code below for your implementation references

Example Code:

//Server.js

...

const { auditLoggerMiddleware } =
require("./components/middleware/auditLoggerMiddleware");

...

app.use(auditLoggerMiddleware);

...

`auditLoggerMiddleware` was added to `index.js` globally, before the routes (and before the rate limiter, so even a 429 gets logged), so it runs for every request — authenticated or not, successful or not.

```
const { auditLoggerMiddleware } = require("./components/middleware/auditLoggerMiddleware");
...
app.use(auditLoggerMiddleware);
app.use(apiLimiter);
...
app.use(`${API_V1}/logs`, logRoutes); // GET /api/v1/logs, Admin only
```

Field	Example	Meaning
level	info / warn / error	Severity, derived from the status code
message	`METHOD path - status`	One-line summary, e.g. GET /api/v1/products - 200
metadata.method / path	String	HTTP verb and the full URL (originalUrl)
metadata.ip, userId, email, role	String / null	Who made the request, or anonymous
metadata.statusCode	Number	The response's HTTP status code
metadata.duration	String	How long the request took, e.g. "42ms"

Verified by performing CRUD as Admin — create, read, update, and delete a category, plus one deliberately failing request (`GET` a category that does not exist, 404) — then calling `GET /api/v1/logs?limit=20` and finding each of those requests in the list, with the 404 logged at `warn` level. The same log entries are visible in MongoDB Atlas → Data Explorer → `logs`.

Output

[Paste screenshot here: 4.1 POST a category (201) through 4.4 DELETE it (200), performed as Admin]

[Paste screenshot here: 4.5 GET a category that does not exist (404)]

[Paste screenshot here: 4.6 GET /api/v1/logs as Admin — the entries above are all present]

[Paste screenshot here: Atlas Data Explorer — logs collection]

Part 5: Creating Input Validation Middleware

Do the following (*Provide the Source code, add more pages if necessary***):**

- We already identify each input needed as per our previous activity
- Inside the validationMiddleware.js, create a input validation based on your UserModel
- Use the following example code for your reference in creating input validation

Example Code:

```
const { body, validationResult } = require('express-validator');

const registerValidation = [
  body('name')
    .isString().withMessage('Name must be a string')
    .isLength({ min: 3 }).withMessage('Name must be at least 3 characters long'),
  body('studentNumber')
    .isNumeric().withMessage('Student number must be a number')
    .isLength({ min: 8, max: 8 }).withMessage('Student number must be exactly 8 digits long'),
  body('course')
    .isString().withMessage('Course must be a string')
    .isLength({ min: 2 }).withMessage('Course must be at least 2 characters long'),
  body('yearLevel')
    .isString().withMessage('Year level must be a string')
    .isLength({ min: 1 }).withMessage('Year level must be at least 1 character long'),
```

```
  body('email')
    .isEmail().withMessage('Invalid email address'),
  body('password')
  .isLength({ min: 6 }).withMessage('Password must be at least 6 characters long'),

  (req, res, next) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() });
    }
    next();
  }
];

module.exports = {registerValidation};
```

The reference example's fields (`name`, `studentNumber`, `course`, `yearLevel`) are from a student-registration form and do not match this project's User model (Activity 2/4: `firstName`, `lastName`, `email`, `password`, plus `role`, which a client is never allowed to set — see Activity 4). The validation rules below are written against the **actual** `userModel.js` fields instead, using the same `express-validator` pattern and the same "collect every error, then reject once" structure as the example.

Errors are returned through this project's existing `sendError`/`HttpStatus.BAD_REQUEST` helper, in the standard `{ success, message }` shape, instead of the example's raw `{ errors: [...] }` — so a validation failure looks exactly like every other 400 response from Activities 2–4, rather than a different shape the client would need to special-case.

components/middleware/validationMiddleware.js

```
const { body, validationResult } = require("express-validator");

const handleValidation = (req, res, next) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return sendError(res, HttpStatus.BAD_REQUEST, errors.array().map((e) => e.msg).join("; "));
  }
  next();
};

const registerValidation = [
  body("firstName").isString().trim().notEmpty().withMessage("First name is required"),
  body("lastName").isString().trim().notEmpty().withMessage("Last name is required"),
  body("email").isEmail().withMessage("Invalid email address").normalizeEmail(),
  body("password").isLength({ min: 6 }).withMessage("Password must be at least 6 characters long"),
  handleValidation,
];

const loginValidation = [
  body("email").isEmail().withMessage("Invalid email address").normalizeEmail(),
  body("password").notEmpty().withMessage("Password is required"),
  handleValidation,
];

const updateUserValidation = [ /* same fields, each .optional() - only changed fields are sent */
];

module.exports = { registerValidation, loginValidation, updateUserValidation };
```

Part 6: Implementing the Input Validation Middleware

Do the following (*Provide the Source code and Output, Add more pages if necessary***):**

- Inside your Routes files import your validationMiddleware.js
- Verify that each request are checked by your input validation middleware
- Perform several functions (CRUD) in your UserRoutes using Postman/Insomnia REST
- Refer to the Example Code below for your implementation references

Example Code:

```
// UserRouter.js
...
const {loginValidation} = require("../middleware/validationMiddleware");
...

router.post("/register", loginValidation, authentication, authorize("Admin"),
UserController.registerUser);

...
```

`validationMiddleware.js` was imported in `userRoutes.js` and `sessionRoutes.js` and placed **before** the controller function, so an invalid request never reaches `signUp`/`signIn`/`updateUser` at all.

```
// userRoutes.js
const { registerValidation, updateUserValidation } = require("../middleware/validationMiddleware");
const { registerLimiter } = require("../middleware/rateLimiterMiddleware");

router.post("/", registerLimiter, registerValidation, signUp);
router.patch("/:id", authentication, updateUserValidation, updateUser);

// sessionRoutes.js
const { loginValidation } = require("../middleware/validationMiddleware");
const { loginLimiter, loginLockout } = require("../middleware/rateLimiterMiddleware");

router.post("/", loginLimiter, loginLockout, loginValidation, signIn);
```

Method	Route	Middleware chain	Note
POST	/api/v1/users	registerLimiter, registerValidation	Validated before signUp ever runs
POST	/api/v1/sessions	loginLimiter, loginLockout, loginValidation	Validated before signIn ever runs
PATCH	/api/v1/users/:id	authentication, updateUserValidation	Each field optional; validated if present

Tested by performing CRUD on `/api/v1/users` in Postman: registering without a `firstName`, with a password under 6 characters, or with an invalid email format each return 400 with a clear message before any database write happens; a valid registration (201) and a valid profile update (200) go through normally.

Output

[Paste screenshot here: Register missing firstName – 400 Bad Request]

[Paste screenshot here: Register with a 3-character password – 400 Bad Request]

[Paste screenshot here: Register with an invalid email – 400 Bad Request]

[Paste screenshot here: Register with a valid body – 201 Created]

[Paste screenshot here: Update own profile with an invalid email – 400 Bad Request]

[Paste screenshot here: Update own profile (valid) – 200 OK]

Part 5: Implementing Helmet Library in your Server

Do the following (*Provide the Source code and Output, add more pages if necessary***):**

- Import the Helmet library by using npm install
- Upon installing import and configure your Helmet Library
- Test your API Security by opening a new terminal and typing "curl -I http://localhost:5000"
- Use the sample code below for your reference

Example Code:

```
// Server.js
const helmet = require("helmet");

...

app.use(helmet({
 contentSecurityPolicy: {
  directives: {
   defaultSrc:     ["'self'"],
```

```
    scriptSrc:      ["'self'"],
    styleSrc:       ["'self'", "'unsafe-inline'"],
    imgSrc:         ["'self'", "data:", "blob:"],
    connectSrc:     ["'self'"],
    fontSrc:        ["'self'", "data:"],
    objectSrc:      ["'none'"],
    frameAncestors:  ["'none'"],
    upgradeInsecureRequests: [],
   },
  },
  frameguard:    { action: "sameorigin" },        // X-Frame-Options: SAMEORIGIN
  referrerPolicy: { policy: "strict-origin-when-cross-origin" }, // Referrer-Policy
}));

...
```

`helmet` was installed (`npm install helmet`) and configured in `index.js`. This backend is a **JSON-only API** — it never renders HTML or serves a script/stylesheet to a browser — so the Content-Security-Policy starts from `default-src: ["'none'"]` (nothing is allowed to load) instead of Helmet's default, which still assumes an app that might serve pages. `frameAncestors: ["'none'"]` and `frameguard: { action: "deny" }` both stop the API from ever being embedded in an `<iframe>` (clickjacking defense, redundant on a JSON API but harmless to keep), and `referrerPolicy` limits what's sent in the `Referer` header on any outgoing link.

index.js

```
const helmet = require("helmet");

app.use(helmet({
  contentSecurityPolicy: { directives: { defaultSrc: ["'none'"], frameAncestors: ["'none'"] } },
  frameguard: { action: "deny" },
  referrerPolicy: { policy: "strict-origin-when-cross-origin" },
}));
```

Tested with `curl -I http://localhost:5000` in a second terminal: the response no longer includes `X-Powered-By: Express` (which used to reveal the framework), and now includes `Content-Security-Policy`, `X-Frame-Options: DENY`, and `X-Content-Type-Options: nosniff`.

Output

[Paste screenshot here: curl -I http://localhost:5000 — security headers present, X-Powered-By gone]

Part 6: Implement CORS Library in your Server

Do the following (*Provide the Source code and Output, add more pages if necessary***):**

- Import the CORS library by using npm install
- Upon installing import and configure your CORS Library
- Use the sample code below for your reference

Example Code:

```
// Server.js
const helmet = require(“helmet”);

...
const corsOptions = {
 origin: “http://localhost:5000/”,
 credentials: true,
 methods: ["GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS"],
 allowedHeaders: ["Origin", "X-Requested-With", "Accept", "Content-Type", "Authorization"],
};

app.options(corsOptions));
app.use(cors(corsOptions));
...
```

`cors` was already installed since Activity 1 (`app.use(cors())`, which allowed every origin). The reference example's `origin: "http://localhost:5000/"` is the **API's own address**, not a frontend's — copying it literally would make the API only callable from itself, which would block the real React frontend (`marzan-client`) entirely. Instead, the allowed origin(s) are read from a new `CORS_ORIGIN` environment variable (`.env`), defaulting to the Vite dev server's port (`http://localhost:5173`) that `marzan-client` runs on; the deployed frontend's Vercel URL gets added here once that is live.

index.js

```
const { CORS_ORIGIN } = require("./components/config/config"); // .env:
CORS_ORIGIN=http://localhost:5173,...

app.use(cors({
  origin: CORS_ORIGIN,
  credentials: true,
  methods: ["GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"],
  allowedHeaders: ["Origin", "X-Requested-With", "Accept", "Content-Type", "Authorization"],
}));
```

Tested in Postman by setting a custom `Origin` header (Postman is not itself subject to CORS, but the server still reflects or omits the header the same way a browser would check it): a request with `Origin: http://localhost:5173` gets back `Access-Control-Allow-Origin: http://localhost:5173`; a request with `Origin: http://evil.example` gets no such header at all, which is what makes a real browser block that response from being read by a script running on the disallowed site.

Output

[Paste screenshot here: Request with an allowed Origin header — Access-Control-Allow-Origin is present]

[Paste screenshot here: Request with a disallowed Origin header — no Access-Control-Allow-Origin header]

Part 7: Creating your Rate Limiter Middleware

Do the following (*Provide the Source code, add more pages if necessary***):**

- Identify what function need rate limiter for possible excessive users request
- Inside the rateLimterMiddleware.js, create a rateLimiter based on your UserModel that requires rate limiter
- Use the following example code for your reference in creating input validation

Example Code:

```
const rateLimit = require("express-rate-limit");

const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 20,
  standardHeaders: true,
  legacyHeaders: false,
  message: { message: "Too many login attempts. Please try again after 15 minutes." },
});
```

```
module.exports = {loginLimiter};
```

Which functions need a rate limiter? Any endpoint an attacker could call repeatedly to gain something — guess a password, create junk accounts, or simply overload the server — needs one. Login is the clearest case (an attacker who can try unlimited passwords will eventually guess one), registration is a close second (spam accounts, or using failed registrations to find out which emails already exist), and a general limit on the whole API protects against plain flooding of any endpoint.

Route	Limit	Limiter	Why
POST /api/v1/sessions (login)	20 / 15 min per IP	loginLimiter	The function an attacker gains the most from — credential stuffing / brute force
POST /api/v1/users (register)	10 / hour per IP	registerLimiter	Prevents spam account creation and probing which emails are already taken
Every route	300 / 15 min per IP	apiLimiter	A general ceiling so no single client can flood the whole API

components/middleware/rateLimiterMiddleware.js

```
const rateLimit = require("express-rate-limit");

const handler = (message) => (req, res) => sendError(res, HttpStatus.TOO_MANY_REQUESTS, message);

const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, limit: 300, standardHeaders: true, legacyHeaders: false,
  handler: handler("Too many requests. Please try again later."),
});

const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, limit: 20, standardHeaders: true, legacyHeaders: false,
  handler: handler("Too many login attempts. Please try again after 15 minutes."),
});

const registerLimiter = rateLimit({
  windowMs: 60 * 60 * 1000, limit: 10, standardHeaders: true, legacyHeaders: false,
  handler: handler("Too many accounts created from this address. Please try again after an hour."),
});

module.exports = { apiLimiter, loginLimiter, registerLimiter, ... }; // loginLockout: see Part 9
```

`handler` is used instead of the example's `message` option so a 429 still comes back in this project's standard `{ success, message }` shape, and `HttpStatus.TOO_MANY_REQUESTS` (429) was added to `config/constants.js` to name it.

Part 8: Implementing Rate Limiter Middleware

Do the following (*Provide the Source code and Output, Add more pages if necessary***):**

- Inside your Routes files import your rateLimiterMiddleware.js
- Verify that each request is being rate limited
- Perform several functions (CRUD) in your UserRoutes using Postman/Insomnia REST

- Refer to the Example Code below for your implementation references

Example Code:

```
// UserRoutes.js
...

const {loginLimiter} = require("../middleware/rateLimiterMiddleware.js");

...

router.post("/login", loginLimiter, UserController.loginUser);

...
```

`apiLimiter` is applied globally in `index.js` (right after the audit logger, so a 429 is still logged); `loginLimiter` and `registerLimiter` are applied to their specific routes.

```
// index.js
app.use(auditLoggerMiddleware);
app.use(apiLimiter);
...
// sessionRoutes.js
router.post("/", loginLimiter, loginLockout, loginValidation, signIn);
// userRoutes.js
router.post("/", registerLimiter, registerValidation, signUp);
```

Tested in Postman: every response carries `RateLimit-Limit` and `RateLimit-Remaining` headers (standard headers from `express-rate-limit`), and `RateLimit-Remaining` goes down by 1 on each request from the same IP, proving the counter is live. CRUD on `/api/v1/users` (register, get profile, update, and — as Admin — delete) all still work normally while under the limit.

Output

[Paste screenshot here: GET / — RateLimit-Limit: 300, RateLimit-Remaining: 299]

[Paste screenshot here: GET / again — RateLimit-Remaining: 298 (counting down)]

[Paste screenshot here: Register → login → update → delete a user — all succeed while under the limit]

Part 9: Enhancement Activity

Do the following:

- Analyze your previous routes and determine what functions need to be implemented with a Rate Limiter and Input Validation Middleware
- Suppose a user try to login multiple times but failed each try put a timer that increases per each login failed attempts (ex. Failed Login: 5x (3 Minutes per try), 8x (5 Minutes per try), 10x (30 Minutes per try)Provide Screenshots of your Source Code and Output, add more pages if necessary

The table below extends the same "which functions need which middleware" analysis used for authorization in Activity 4, this time for validation and rate limiting.

Route	Needs	Why
POST /api/v1/sessions	Validation + Rate limiter + Lockout	Attacker-facing; wrong credentials and brute force both need to be contained
POST /api/v1/users	Validation + Rate limiter	Public write endpoint; spam accounts and bad data both need blocking
PATCH /api/v1/users/:id	Validation	Already behind authentication + the self-or-admin check (Activity 4); only the shape of the data needs checking
POST /api/v1/products, /categories, /suppliers	Validation (candidate)	Already Admin-only (Activity 4); extending the same validation pattern here is a natural next step, out of this activity's stated scope ("based on your UserModel")
Every other route	apiLimiter only	No extra user input to validate (GET requests, or protected by auth/RBAC already)

Escalating login lockout

On a wrong password, the login route now increases a per-account lockout instead of only relying on the IP-based `loginLimiter`: **5 failed attempts → locked 3 minutes, 8 → 5 minutes, 10+ → 30 minutes.** It is keyed by email (not IP), so the same account stays protected even if the attacker switches IP addresses, and it is a simple in-memory `Map` (not a new database field) — reasonable for a single-instance lab project, and noted here as the tradeoff: it resets if the server restarts, and a multi-instance deployment would need a shared store (e.g. Redis) instead.

```
const failedLogins = new Map(); // email -> { count, lockedUntil }

const lockoutDuration = (failCount) => {
  if (failCount >= 10) return 30 * 60 * 1000;
  if (failCount >= 8) return 5 * 60 * 1000;
  if (failCount >= 5) return 3 * 60 * 1000;
  return 0;
};

const loginLockout = (req, res, next) => {
  const email = String(req.body.email || "").toLowerCase();
  const entry = failedLogins.get(email);
  if (entry?.lockedUntil && entry.lockedUntil > Date.now()) {
    const minutes = Math.ceil((entry.lockedUntil - Date.now()) / 60000);
    return sendError(res, HttpStatus.TOO_MANY_REQUESTS, `Too many failed login attempts. Try again
in ${minutes} minute(s).`);
  }
  next();
};

// called from userController.signIn:
// recordLoginFailure(email) on a wrong password, recordLoginSuccess(email) on success
```

Tested by failing to log in 5 times in a row with the correct email and a wrong password (each returns 401, as a normal wrong-password attempt does), then a 6th attempt — even with the **correct** password — returns **429 "Too many failed login attempts. Try again in 3 minute(s)."**, proving the account is locked regardless of whether the right password is finally used.

Output

[Paste screenshot here: Failed attempts 1–5 — each 401 Unauthorized]

[Paste screenshot here: 6th attempt with the correct password — 429, locked for 3 minutes]

Conclusion

The Bulldogs Exchange API now logs every request to the database, validates registration/login/profile-update input before it reaches a controller, and is hardened with Helmet, a restricted CORS origin list, and layered rate limiting — a general per-IP ceiling, tighter limits on login and registration, and an escalating per-account lockout on repeated failed logins. All of this sits in front of the authentication, RBAC, and RESTful API built in Activities 3–4, which continued to work correctly underneath it, confirmed with Postman across every scenario: valid requests, invalid input, missing or wrong-role tokens, disallowed origins, and requests over the rate limit.

Activity Rubrics

Criteria	Beginning (1)	Developing (2)	Proficient (3)	Excellent (4)	Pts
Audit Logging and Custom Middleware	Audit logging middleware was not implemented or failed to record API activities.	Audit logging was partially implemented but recorded incomplete or inaccurate request information.	Audit logging correctly records the required API activities with only minor issues.	Audit logging is fully implemented and accurately records relevant request and response information.	
Request Validation and Error Handling	Request validation and error handling were not implemented or failed to handle invalid requests properly.	Validation and error handling were partially implemented but several invalid requests were not handled appropriately.	Required validation rules and error handling function correctly with only minor issues.	Comprehensive request validation and error handling are correctly implemented, producing clear and consistent responses without causing application failures.	
API Security Configuration	Helmet, CORS, or rate limiting were not implemented or were incorrectly configured.	Some security features were implemented, but configurations contained significant issues or were ineffective.	Helmet, CORS, and rate limiting were correctly configured with only minor issues.	Helmet, CORS, and rate limiting are properly configured according to the application's requirements and effectively applied.	
Middleware Integration and API Functionality	Middleware was poorly integrated, causing major application errors or preventing API	Middleware was partially integrated, but execution order or interaction	Middleware components were properly integrated and existing authentication,	Middleware was systematically integrated with an appropriate execution order,	

	functionality.	between middleware components caused several issues.	authorization, and API functionality continued to work with minor issues.	while all existing authentication, RBAC, and API functionality remained fully operational.	
Testing, Documentation, and Submission	Required tests, screenshots, and documentation were largely missing or incomplete.	Some middleware and security scenarios were tested, but several required tests or documentation were missing.	Required middleware, validation, error handling, and security scenarios were tested and documented with only minor omissions.	All required scenarios were thoroughly tested using Postman, results were clearly demonstrated, and all screenshots, source code, and documentation were complete and well-organized.	
Total Score					